Version: 28092023

SQL Queries

Overview

With the advent of a data-driven world, the amount of data continues to increase, many businesses store their information inside a large relational database. Excel Spreadsheet is good for presentations and small analyses but it is inadequate for Big Data analysis.

In this course, you will learn basic database language and the mechanism of database. At the end of the course, you will be capable of retrieving data from databases and discovering insights with handy analysis tools as a data analyst.

Unit 1

What is SQL

What will you learn

- What is SQL
- What is the difference between programming language and query language
- What is the difference between SQL and Excel

What is SQL

- SQL is "**Standard Query Language**", which is a way to **talk to tables**
- SQL is NOT a programming language
- In programming languages, we can use variables, loops and functions to build logic
- In query languages, we can only use a limited set of instructions since the logic of fetching data is pre-programmed
- We can directly open Excel files with Excel application, but we need to connect to the remote storage to view the data in it

Unit 2

Select

What will you learn

- SELECT columns
- create new column by calculations
- create new columns by built-in function
- CASE-WHEN command to return value based on conditions

Select rows and columns

- To select all rows and all columns from the table, we use SELECT:

```
SELECT *
FROM exam
```

- To select particular columns, we modify the * to the column names you want:

```
SELECT
   exam_id, subject_id, student_id, marks
FROM exam
```

Filter rows

- To filter rows with value, we use WHERE:

```
SELECT *
FROM exam
WHERE subject_id = 1
```

- We can filter rows with multiple conditions with AND and OR operators:

```
SELECT *
FROM exam
WHERE subject_id = 1 AND marks > 70
```

Like

- The LIKE operator returns TRUE if its first text argument matches the wildcard pattern in its second argument.
- It is commonly used in a WHERE clause to select rows based on a column matching a given string pattern:

```
SELECT *
FROM exam
WHERE marks LIKE '7%'
```

- The pattern is made up of a string that includes the following wildcards:
  - The percent character %: matches zero to any number of arbitrary characters.
  - The underscore character _: matches a single arbitrary character

Create new column by calculations

- We can include arithmetic in the select part:

```
SELECT
    exam_id, subject_id, student_id, marks / 2
FROM exam
```

Create new column by built-in function

- In the following example we used the lower() built in function to change texts to lower case:

```
SELECT
    exam_id,
    exam_description,
    LOWER(exam_description)
FROM exam
```

- Give the calculated column a name so that it has a better column name. Note that the as keyword is optional:

```
SELECT
    exam_id,
    exam_description,
    LOWER(exam_description) as lower_exam_description
FROM exam
```

SQL Functions

- Besides lower(), there are a lot of other functions for us to transform values.

- Here's some examples:
  - LOCATE(): Returns the position of the first occurrence of a substring in a string
  - SIGN(): Returns the sign of a number (positive or negative)
  - HOUR(): Extract the hour part from a given date

Dialect

- Each kind of database provides their own set of functions. We call each of the different kind a SQL Dialect. Therefore, make sure you are looking at the correct documentation

- E.g. When you are developing MySQL syntax, do NOT search SQL Server in browser: SQL Server is made by Microsoft.

Case-when

- One special function to replicate if-else statement in Python is case-when:

```
select
    *,
    case
        when marks > 70 then 'Excellent'
        when marks > 60 then 'Good'
        when marks > 50 then 'Pass'
        else 'Failed'
    end
from exam
```

SQL String Literal

- Single quote(') is used to denote strings in mostly all SQL dialects.

- However, note that single quote ('), double quote (") and backtick (`) have different meanings, and generally not interchangeable (Unlike in Python).

Unit 3

Sorting

What will you learn

- sort data with ORDER BY command
- sort data with ascending or descending order
- sort data by multiple fields
- sort data by multiple fields, different directions

Order by

- We can sort by a field in ascending order:

```
SELECT *
FROM exam
ORDER BY marks
```

- We can also specify a descending order by the desc keyword. Note that we can specify an ascending order using the asc keyword, but since ascending order is the default mode, we may omit that.

```
SELECT *
FROM exam
ORDER BY marks desc
```

```
SELECT *
FROM exam
ORDER BY marks asc
```

Sort by multiple fields

- We can sort by multiple fields. In the following example, we first order the rows by student_id, then order by marks within the rows with the same student_id:

```
SELECT *
FROM exam
ORDER BY student_id, marks
```

- In multiple fields sorting, we can specify one field to be asc and one field to be desc:

```
SELECT *
FROM exam
ORDER BY student_id asc, marks desc
```

Unit 4

Aggregation

What will you learn

- common methods such as sum, mean, max, min, median
- count number of rows and non-null values for a column

Aggregation methods

- There are quite a lot of aggregation methods built-in in SQL. For example:
  - Sum
  - Mean
  - Max
  - Min
  - Median or Percentile (A difficult one, not many engines natively support)
- For example:

```
SELECT
    avg(marks),
    max(marks),
    min(marks)
FROM
    Exam
```

Count

- Count() is a special aggregation method:
  - When we do avg(marks), we calculate from the marks column.
  - Count number of rows, we do not need the marks column.

- For example, count() or count(*) or count(1) all are counting number of rows no matter it contains null or non-null values:

```
SELECT
    count(), count(*), count(1)
FROM
    Exam
```

Count non-null values

- Count with a column name is equivalent to count non-null values in the column.

- Note the difference between count(), count(marks) and count(exam_id) in below example:

```
SELECT
    count(),
    count(marks)
FROM
    Exam
```

Unit 5

Grouping

What will you learn

- remove duplicates with DISTINCT
- summarize records belong to groups with GROUP BY

Deduplication with Distinct

- The following example remove duplicates in the student_id column:

```
select distinct student_id
from exam
```

- You can also de-duplicate on multiple columns. It returns all the unique combinations of the fields:

```
select distinct student_id, exam_id
from exam
```

Group by

- Group by summarize records that belongs to a group. Similar to distinct: Group by first find out all the unique combinations, then perform aggregation on the rest of the columns:

```
select
    student_id, exam_id
from
    exam
group by
    student_id, exam_id
```

```
select
    exam_id, subject_id, avg(marks)
from
    exam
group by
    exam_id, subject_id
```

Having

- Similar to WHERE, Having is used with aggregate functions (e.g. COUNT(), MIN(), MAX(), SUM(), AVG()) to filter the groups.

- HAVING statement always comes after GROUP BY, but before ORDER BY

```
select
    exam_id, avg(marks),
max(marks), min(marks)
from
    exam
group by
    exam_id
having avg(marks) > 65
```

Unit 6

Join tables

What will you learn

- handling unmatched rows with INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN and CROSS JOIN

INNER JOIN or simply JOIN

- The INNER JOIN keyword selects records that have matching values in both tables.

```
SELECT *
FROM conduct
INNER JOIN student ON conduct.student_id =
student.student_id
```

LEFT JOIN

- The LEFT JOIN keyword returns all records from the left table (table1), and the matching records from the right table (table2). The result is 0 records from the right side, if there is no match.

```
SELECT *
FROM conduct
LEFT JOIN student ON conduct.student_id =
student.student_id
ORDER BY conduct.student_id
```

RIGHT JOIN

- The RIGHT JOIN keyword returns all records from the right table (table2), and the matching records from the left table (table1). The result is 0 records from the left side, if there is no match.

```
SELECT *
FROM conduct
RIGHT JOIN student ON conduct.student_id =
student.student_id
ORDER BY conduct.student_id
```

FULL JOIN

- The FULL OUTER JOIN keyword returns all records when there is a match in left (table1) or right (table2) table records.

```
SELECT *
FROM conduct
FULL JOIN student ON conduct.student_id =
student.student_id
ORDER BY conduct.student_id
```

Cross Join

- It creates every combinations of rows from each table with NO joining condition. Such joining does not consider any join keys

```
Select *
from teacher
cross join subject
```

Unit 7

Nested Query

What will you learn

- nesting a single value
- nesting a table

Nesting a single value

- In below example, one query is nested in another table.

```
SELECT *
FROM student
WHERE weight > (
    SELECT avg(weight)
    FROM student
)
```

Nesting a table

- In below example, we wish to list out all the students that are heavier than the average weight in each class.

Step 1: Calculate the average weight for each class

```
SELECT class_id, avg(weight)
FROM student
GROUP BY class_id
```

Nesting a table

- In below example, we wish to list out all the students that are heavier than the average weight in each class.

Step 2: LEFT JOIN the tables (student and avg_weight) with student.class_id = avg_weight.class_id

```
SELECT *
FROM student
LEFT JOIN (
    SELECT
        class_id,
        avg(weight) avg_weight
    FROM student
    GROUP BY class_id
) avg_weight_table on student.class_id = avg_weight_table.class_id
```

Nesting a table

- In below example, we wish to list out all the students that are heavier than the average weight in each class.

Step 3: Filter rows with WHERE and student.weight > avg_weight_table.avg_weight

```
SELECT *
FROM student
LEFT JOIN (
    SELECT
        class_id,
        avg(weight) avg_weight
    FROM student
    GROUP BY class_id
) avg_weight_table on student.class_id = avg_weight_table.class_id
WHERE student.weight > avg_weight_table.avg_weight
```

Nesting a table

- In below example, we wish to list out all the students that are heavier than the average weight in each class.

Step 4: Select only columns from student table but not avg_weight_table

```
SELECT student.*
FROM student
LEFT JOIN (
    SELECT
        class_id,
        avg(weight) avg_weight
    FROM student
    GROUP BY class_id
) avg_weight_table on student.class_id = avg_weight_table.class_id
WHERE student.weight > avg_weight_table.avg_weight
```

WITH-AS

- We can also store a table in a variable for later use with AS command:

Store in a variable as avg_weight_table:

```
with avg_weight_table as (
    SELECT
        class_id,
        avg(weight) avg_weight
    FROM student
    GROUP BY class_id
)
```

Use it:

```
SELECT student.*
FROM student
LEFT JOIN avg_weight_table on student.class_id = avg_weight_table.class_id
WHERE student.weight > avg_weight_table.avg_weight
```